

Week 1 Getting Started Guide - Flutter Setup, Run, Build

Table of Contents

Week 1 — Getting Started Guide	1
What you need	2
Part 1 — Install the toolchain	2
Part 2 — Verify with flutter doctor	3
Part 3 — Set up a device to run on	4
Part 4 — Get the sample project	4
Part 5 — Run the app	5
Part 6 — Understand the project structure	5
Part 7 — Build the app	6
Part 8 — The Git workflow	6
Part 9 — Using AI tools (the right way)	7
Part 10 — Troubleshooting	7
Appendix — Command cheat sheet	8
What's next	9

Week 1 — Getting Started Guide

Installing Flutter, Running, and Building the FieldLog Sample Project

Cross-Platform Mobile Application Development Master's in IT Engineering · Asia Euro
University Lecturer: **Mr. Sok Pongsametrey** · metreysk@gmail.com · 017 856 900 ·
<https://pongsametrey.com>

This is a bonus reference to help you set up your environment and run the Week 1 sample project. Keep it open beside your laptop during the first session. If you complete everything here before Week 1 begins, you will get the full benefit of the live-coding session instead of spending it on setup.

What you need

Tool	Why	Where
Flutter SDK (latest stable)	The framework and command-line tools	docs.flutter.dev/get-started/install
Android Studio	Android SDK, emulator, and the easiest Flutter setup	developer.android.com/studio
VS Code or Cursor (optional)	Lighter editor; many prefer it for daily coding	code.visualstudio.com/ / cursor.com
Git	Version control — used from day one	git-scm.com
An AI coding assistant	Required in this course (Claude / Copilot / Cursor)	claude.ai/ / github.com/features/copilot
A GitHub account	Where your work lives	github.com

You do **not** need a Mac or an Apple Developer account. This course targets Android for all hands-on work; iOS is covered conceptually.

Part 1 — Install the toolchain

1.1 Install the Flutter SDK

Follow the official guide for your operating system at **docs.flutter.dev/get-started/install**. The summary:

Windows

1. Download the Flutter SDK zip.
2. Extract it to a folder *without spaces or special characters* — for example, `C:\dev\flutter` (do **not** put it in `C:\Program Files`).
3. Add `C:\dev\flutter\bin` to your Path environment variable.
4. Open a new terminal and run `flutter --version`.

macOS / Linux

1. Download and extract the Flutter SDK, e.g. to `~/development/flutter`.
2. Add it to your PATH in `~/.zshrc` or `~/.bashrc`:
`export PATH="$PATH:$HOME/development/flutter/bin"`
3. Open a new terminal and run `flutter --version`.

1.2 Install Android Studio

1. Download and install Android Studio.

2. On first launch, complete the setup wizard — it installs the Android SDK, command-line tools, and an emulator image.
3. Open **Settings** → **Plugins**, search for **Flutter**, and install it (this also installs the Dart plugin). Restart when prompted.

1.3 Install Git

- **Windows:** download from git-scm.com and install with defaults.
- **macOS:** git comes with Xcode command-line tools — run `xcode-select --install`, or install via Homebrew: `brew install git`.
- **Linux:** `sudo apt install git` (Debian/Ubuntu).

Verify: `git --version`.

1.4 Configure Git with your identity

```
git config --global user.name "Your Full Name"
git config --global user.email "your-email@example.com"
```

Use your real name — this is your professional record.

1.5 Install an AI coding assistant

Pick one (any one is acceptable for this course):

- **Claude** — claude.ai (web) or the desktop app
- **GitHub Copilot** — install the extension in VS Code or Android Studio
- **Cursor** — a VS Code-based editor with AI built in

Verify it works by asking it: *“Explain the difference between `final` and `const` in Dart.”*

Part 2 — Verify with flutter doctor

`flutter doctor` is the single most important diagnostic command. Run it:

```
flutter doctor
```

You want the **Flutter** and **Android toolchain** lines to show a green check (✓). A typical healthy output:

```
[✓] Flutter (Channel stable, 3.x.x)
[✓] Android toolchain - develop for Android devices
[✓] Android Studio (version 2025.x)
[✓] Connected device (1 available)
[✓] Network resources
```

If the Android toolchain shows an issue about licenses, run:

```
flutter doctor --android-licenses
```

and accept all of them.

The **Xcode** line will show a red X on Windows and Linux — that is expected and fine. We do not build for iOS in this course.

Part 3 — Set up a device to run on

You need either an emulator or a physical Android phone.

3.1 Option A — Android emulator

1. In Android Studio, open **Device Manager** (the phone icon in the toolbar).
2. Click **Create Device**.
3. Pick a phone (e.g. Pixel 7) → **Next**.
4. Select a system image (a recent API level, e.g. API 34) → download it if needed → **Next** → **Finish**.
5. Press the ► play button to launch the emulator.

3.2 Option B — Physical Android phone

1. On your phone, go to **Settings** → **About phone** and tap **Build number** seven times to enable Developer Options.
2. In **Settings** → **Developer Options**, enable **USB debugging**.
3. Connect your phone to your computer with a USB cable.
4. Accept the “Allow USB debugging?” prompt on the phone.
5. Confirm the device is detected: `flutter devices`.

A physical phone is faster than an emulator and is the recommended way to demo your work.

Part 4 — Get the sample project

The lecturer will share a GitHub link to the FieldLog sample repository. Once you have it:

```
# Clone the sample project
git clone <repo-url-from-lecturer> fieldlog_flutter
cd fieldlog_flutter
```

This repository contains the Dart source and configuration only. The platform folders (android/, ios/) are generated by Flutter and are intentionally not committed (they are large and environment-specific). Generate them once:

```
# Scaffold the platform folders around the existing lib/ – one time only
flutter create . --project-name fieldlog_flutter --platforms=android
```

```
# Download the package dependencies
flutter pub get
```

flutter create . is safe to run here: it adds the missing android/ folder and other scaffolding **without overwriting** your existing lib/, pubspec.yaml, or test/.

Part 5 — Run the app

With an emulator running or a phone connected:

```
flutter run
```

You should see the FieldLog welcome screen: the app name, a short description, and a **Get started** button. Tapping the button shows a SnackBar that says “Welcome to FieldLog!”

Hot reload and hot restart

While flutter run is active, in the terminal:

- Press **r** for **hot reload** — applies code changes in under a second, keeping app state.
- Press **R** for **hot restart** — restarts the app from scratch (loses state).
- Press **q** to quit.

Hot reload is one of Flutter’s best features. Change a string in onboarding_screen.dart, save, press **r**, and watch it update instantly.

Part 6 — Understand the project structure

```
fieldlog_flutter/
├── pubspec.yaml           ← dependencies, SDK pin, assets
├── analysis_options.yaml  ← lint rules (enforces our OOP rules)
├── .gitignore            ← excludes build/, .dart_tool/, keystores
├── lib/
│   ├── main.dart         ← runApp() ONLY – 4 lines
│   └── src/
│       ├── app.dart      ← FieldLogApp: title, theme, first route
│       └── features/
│           └── onboarding/
│               └── presentation/
│                   └── onboarding_screen.dart ← the first screen
└── test/
    └── widget_test.dart  ← starter tests
```

The most important rule: **all your code lives under lib/src/features/<feature>/**. You will create new feature folders (profile, logs, sync, ...) in the weeks to come. `main.dart` never grows beyond its 4 lines.

Part 7 — Build the app

There are three builds you will use across the course.

7.1 Debug build (development)

This is what `flutter run` produces. Fast to build, includes debugging tools, not optimized. Use it all semester for development.

7.2 Release APK (a single installable file)

```
flutter build apk --release
```

The output appears at `build/app/outputs/flutter-apk/app-release.apk`. You can copy this `.apk` to any Android phone and install it directly. Useful for quick sharing.

7.3 Release App Bundle (for the Play Store)

```
flutter build appbundle --release
```

The output appears at `build/app/outputs/bundle/release/app-release.aab`. This is the format Google Play requires. We will sign and upload this in **Week 13**.

Note: in Week 1 a release build works without signing configuration because Flutter uses a debug key by default. Real signing for the Play Store comes in Week 13.

Part 8 — The Git workflow

You use Git from the very first exercise. The rhythm:

```
# See what changed
```

```
git status
```

```
# Stage your changes
```

```
git add .
```

```
# Commit with the course convention: E[WW].[N]: <action>
```

```
git commit -m "E01.1: FieldLog skeleton with feature folders"
```

```
# Push to your GitHub repo
```

```
git push
```

Commit message convention

Every commit message starts with the exercise number, then a short action:

- E01.1: initial FieldLog skeleton
- E02.1: add go_router and profile route
- E04.1: replace setState with Riverpod notifier

This makes your `git log` a readable history of the course. You will be asked to show it during the mid-term.

.gitignore comes first

The very first thing you commit is `.gitignore`. It prevents the large `build/` and `.dart_tool/` folders — and any keystores — from ever entering your history. If you accidentally commit them, they are painful to remove later.

Part 9 — Using AI tools (the right way)

AI assistants are **required** in this course. Two rules:

1. **Disclose.** Every exercise README has an “AI Notes” section: what you prompted, what the AI gave you, what you changed and why.
2. **Understand.** If you cannot explain your own code when called on in class, the exercise is not credited.

Good AI prompts for Week 1

- “My flutter doctor shows <paste exact error>. I am on <OS>. How do I fix it?”
- “Explain what each line of this generated `main.dart` does.”
- “Is `flutter_lints: ^4.0.0` the current stable version? Check `pub.dev`.”

What AI gets wrong (verify these)

- **Outdated package versions** — AI training data lags. Always check `pub.dev`.
 - **Made-up APIs** — AI sometimes invents method names. If it does not autocomplete in your IDE, it probably does not exist.
 - **Single-file `main.dart`** — AI loves to put everything in one file. That violates our structure. Always split into feature folders.
-

Part 10 — Troubleshooting

Symptom	Likely cause	Fix
<code>flutter: command</code>	Flutter not on PATH	Re-check Part 1.1; open a NEW terminal

Symptom	Likely cause	Fix
not found		after editing PATH
flutter doctor Android toolchain issue	Licenses not accepted	flutter doctor --android-licenses, accept all
No devices found	Emulator not running / phone not in debug mode	Start the emulator, or enable USB debugging (Part 3)
App builds but shows a blank/white screen	Error during startup	Check the terminal output for a red stack trace
Khmer text shows as boxes on emulator	Emulator lacks a Khmer font	Install a Khmer font in the emulator's system settings, or test on a physical phone
Gradle task assembleDebug failed	Android build config issue	Run flutter clean, then flutter pub get, then retry. If it persists, paste the exact error into your AI assistant
Changes do not appear	Forgot hot reload	Press r in the flutter run terminal, or R for a full restart
pub get fails with version conflict	Incompatible package versions	Check the conflicting package on pub.dev for a compatible version
flutter clean is a reset, not a fix. If flutter clean “solves” your problem, there may still be a real bug hiding — note what happened and ask in class.		

Appendix — Command cheat sheet

Command	What it does
flutter --version	Show the installed Flutter version
flutter doctor	Diagnose your environment
flutter doctor --android-licenses	Accept Android SDK licenses
flutter create <name>	Create a new Flutter project
flutter create .	Add platform folders to an existing project
flutter pub get	Download dependencies
flutter devices	List connected devices and emulators
flutter run	Run the app in debug mode
flutter run -d <device-id>	Run on a specific device
flutter test	Run all tests
flutter analyze	Run the linter / static analysis
flutter build apk --release	Build a release APK

Command	What it does
<code>flutter build appbundle --release</code>	Build a release App Bundle (Play Store)
<code>flutter clean</code>	Delete build artifacts (reset)
<code>r / R</code> (in <code>flutter run</code>)	Hot reload / hot restart

What's next

Once you can run the FieldLog sample and see the **Get started** button working, you are ready for Week 1. In the session we build this skeleton together from scratch (exercise **E01.1**), then push it to your own GitHub repository.

Bring your laptop charged. We code live every week.

This guide is part of the Week 1 deliverable set: Lesson Plan, Slides, QCM Bank, the FieldLog sample project, and this Getting Started Guide.